

# FIRE\_Modelica

멀티로터의 강체 운동, 추진계, Chassis·고정 payload, 접촉 전환, 네 종류의 sampled sensor를 조립하는 Modelica 라이브러리입니다. FIRE\_Modelica\_Architecture\_Proposal.md의 초기 구현 범위를 적용했습니다. 기존 fixed wing·rover 예제와 센서는 호환 경로로 유지합니다.

라이브러리 디렉터리와 정식 namespace는 모두 FIRE\_Modelica입니다. IDE나 외부 빌드 설정도 새 FIRE\_Modelica/package.mo 경로를 사용해야 합니다. \_Update 보조 namespace 역시 FIRE\_Modelica\_Update로 통일했습니다.

## 시작하기

OpenModelica와 Modelica Standard Library 4.0.0이 필요합니다. 저장소의 package.mo를 열고 다음 예제를 실행합니다.

| 예제                            | 내용                                       |
|-------------------------------|------------------------------------------|
| Examples.QuadHover            | 4-arm/4-rotor의 초기 추력 평형                  |
| Examples.QuadImuOnly          | SensorSuite 없이 plant에 IMU를 직접 연결하는 실험 구성 |
| Examples.QuadImuResponse      | 같은 IMU 조립체 내부 Response를 채널별 1차 응답으로 교체   |
| Examples.HexaHover, OctoHover | 같은 운동방정식으로 6/8 rotor 조립                  |
| Examples.CoaxialHover         | 4-arm/8-rotor 배치; rotor 간 유동 간섭은 제외      |
| Examples.QuadDrop             | 4개 leg의 독립적인 지면 접촉과 정착                   |
| Examples.FixedPayload         | 편심 payload에 의한 질량·CG·관성·mount 변화         |

Hover 예제는 rotor를 평형 속도로 초기화한 **open-loop 검증 모델**입니다. 비행 제어기나 실제 기체의 보정된 parameter set은 포함하지 않습니다.

저장소 루트에서 검증과 구성 생성을 실행할 수 있습니다.

```
python3 tools/verify.py
python3 tools/generate_config.py configs/hex.toml --check
FIRE_TEST_OMC=1 python3 -m unittest discover -s tools -p test_generate_config.py -v
python3 tools/probe_export.py
```

Python 3.11 이상은 표준 tomllib을 사용합니다. Python 3.10에서는 toml이 필요합니다. Native 검증과 구성 생성의 결과는 ignored build/에 저장됩니다. FMU 실행 범위와 제한은 [export 검증 기록](#)을 확인하세요.

생성 모델은 Vehicles.Copter.MultirotorWithSensors를 상속하며 demand[nActuators] 입력을 유지합니다. 각 demand는 \*\* 목표 rotor 속도의 비율 [0,1]\*\*이고 추력 비율이 아닙니다. .mo와 함께 생성되는 manifest는 source hash, compiler, parameter와 선언된 capability를 기록합니다. --check는 방정식 검사이며 simulation 검증을 뜻하지 않습니다.

## 패키지 구성

|                                 |                                              |
|---------------------------------|----------------------------------------------|
| Interfaces/                     | truth, measurement, legacy bus 계약            |
| Utilities/Math/                 | quaternion·회전·기하 함수                          |
| Data/                           | 질량 record, geometry, Quad/Hex/Octo/X8 preset |
| Physical/Mechanical/            |                                              |
| Dynamics/                       | 공통 quaternion 6DOF                           |
| Chassis/{Core,Arms,Payloads}/   | 강체 부품의 질량·CG·관성                              |
| Chassis/LandingGear/            | massless compliant point leg 조립              |
| Contact/                        | unilateral spring/damper와 마찰                 |
| Aerodynamics/{Blades,...}/      | blade 공력과 body drag                          |
| Logical/Communications/         | 기존 PWM/UART/SPI interface                    |
| Systems/{Propulsion,Actuation}/ |                                              |

```
Systems/Sensing/  
{IMU,Magnetometer,GNSS,Barometer}/ Sensor.mo 대표 조립체와 BaseClasses 측정 계약  
ResponseModels/              즉시 전달·채널별 1차 응답  
SensorSuite.mo                기본 네 센서의 선택적 조립  
Worlds/{Environment,Terrain}/  
Vehicles/{Copter,FixedWing,Rover}/  
Adapters/{Legacy,FastDyn}/  
Examples/ Tests/ configs/ tools/ docs/ upstream/
```

Physical.Electrical, Logical device/register/FIFO-driver-scheduler, EMI/vulnerability, dynamic blade flapping은 후속 구현입니다. 초기 비행 모델에는 이를 위한 빈 package나 가짜 동작을 추가하지 않았습니다. Logical.Communications의 Link는 여전히 interface이며 완성된 protocol/channel이 아닙니다.

## 조립과 물리 계약

- World는 ENU, body는 합성 CG 기준 FLU입니다. Quaternion은 {w,x,y,z}, body→world의 Hamilton convention입니다. 내부 단위는 SI입니다.
- geometry.nArms, nRotors, nLegs, nActuators는 독립적인 compile-time parameter입니다. Motor 수를 바꿔도 강체 식을 복사하지 않습니다. 채널은 geometry.actuatorIndex가 지정합니다.
- Mount 위치는 고정 chassis 기준 C에 입력합니다. 합성 CG를 구한 뒤 rotor-leg-IMU-GNSS-barometer 위치를 CG 기준으로 변환합니다. Mount의 회전은 전체 3×3 행렬입니다.
- useAssembledMass=false는 aggregate 총질량·CG·관성만 사용합니다. true는 core, arms, payloads, additionalParts만 합산합니다. Battery·motor 등의 무게는 이 budget에 한 번만 넣습니다. 비어 있지 않은 componentId의 중복은 거부합니다.
- SpeedDrivenRotor는 FirstOrderSpeed와 QuadraticBlade를 조립합니다. Blade는 hub 힘·모멘트를 돌려주며 vehicle이 CG moment arm을 한 번 적용합니다. 외부 wrench는 externalForce\_b, externalMoment\_b로 입력합니다.
- Leg는 질량이 없는 접촉점이고 sprung mass는 기체입니다. 각 leg는 signed gap으로 접촉을 전환합니다. 반력은 음수가 되지 않으며 접촉 시 속도를 재설정하지 않습니다.
- measurements와 truth는 별도 record입니다. IMU에는 specific force와 회전 lever arm 향이 포함됩니다. 각 센서는 독립 주기로 sampling/hold하며 acquisition timestamp를 제공합니다.

상세 parameter-경계-migration은 [구현 설명](#), 센서의 시간 계약은 [Systems/Sensing/README.md](#)에 있습니다.

각 센서는 Systems.Sensing에서 측정·bias·sampling을 담당합니다. SensorSuite는 기본 네 센서의 연결을 제공하며 필수 경유 계층은 아닙니다. 개별 센서 연구나 다중 IMU 구성에서는 Examples.QuadImuOnly처럼 필요한 센서를 직접 배치할 수 있습니다. 각 family의 대표 모델은 Sensor.mo이며, 내부 Measurement와 Response를 redeclare로 교체합니다. 기본 Ideal 응답 또는 채널별 시정수의 FirstOrder 응답을 선택하고 bias·sampling parameter를 설정할 수 있습니다. Examples.QuadImuResponse가 실제 설정 예제입니다. 구조 교체는 컴파일 전 설정이며 실행 중 register 설정 변경은 구현하지 않았습니다. 검증된 누적 구성이 필요해지면 얇은 fidelity preset을 추가할 수 있습니다.

이전 최상위 Sensors 경로는 제거했습니다. 신규 경로의 IMU.MountedSampled와 나머지 Sampled 이름도 각 family의 Sensor로 대체했으며 별칭을 남기지 않았습니다. 기존 LowFidelity는 Systems.Sensing의 각 family로 옮겨졌으며 FixedWing·Rover가 계속 사용합니다. 해당 기체의 물리량·좌표계·입출력을 전환한 뒤 중복 센서 구현을 통합할 예정입니다.

물리 검증 모델과 수치 수용 기준은 [검증 범위](#)에 정리했습니다.

## 기존 코드와 host 연결

정식 namespace는 FIRE\_Modelica입니다. 이전 namespace가 필요하면 정식 package.mo를 먼저 로드하고 compat/FIRE\_Modelica\_Update/package.mo를 추가로 로드합니다. 이전 Communications와 Worlds.Environment에는 호환 별칭이 있습니다. 최상위 Actuators와 그 하위 별칭은 제거했습니다. Servo는 Systems.Actuation.RotaryServo, 기존 PWM 정규화는 Adapters.Legacy.PwmCommandAdapter를 직접 사용합니다. 이전 root namespace를 로드해도 제거한 Actuators 경로가 복원되지는 않습니다.

PwmCommandAdapter는 수치 PWM 명령을 sampling하고 정규화하는 adapter이며 물리 actuator가 아닙니다. FixedWing·Rover는 이 모델을 직접 참조하고 기존 sampling·정규화 동작을 유지합니다. 기체 내부 instance 이름 pwmActuator는 기존 modifier 경로 보

존을 위해 유지했습니다. `Actuators.PwmActuator` 및 `Adapters.Legacy.PwmActuator`를 사용하는 외부 class 선언은 새 이름으로 바뀌어야 합니다. Motor·blade 조립은 `Systems.Propulsion`, servo 등 actuator 조립은 `Systems.Actuation`에 둡니다.

`Adapters.FastDyn.MultirotorValueAdapter`는 N-channel PWM 숫자 입력과 FRD/NED 값 출력을 제공하는 **통합용 경계**입니다. PWM 단위는 microseconds이며 GNSS는 local Cartesian 좌표입니다. 실제 FastDyn backend나 firmware driver에 연결한 결과를 의미하지 않습니다. Ground truth는 ENU/FLU로 별도 출력됩니다.

`Adapters.FastDyn.MultirotorFmu`는 이 adapter를 조립해 FMU용 vector/scalar IO를 노출합니다. `tools/probe_export.py`는 8-rotor·4-leg 구성에서 실제 FMI 2.0 Co-Simulation binary의 입력, 센서, 접촉·이륙을 검사합니다. 이는 OpenModelica export 검증이며 Rumoca 또는 실제 FastDyn 통합의 검증은 아닙니다.